

+++ date = '2026-03-13T20:48:12-07:00' draft = false title = 'Practica4: Programacion logica en Prolog' +++

Reporte de Practica 04: Programacion logica en Prolog

Universidad Autonoma de Baja California
Facultad de Ingenieria, Arquitectura y Diseno

Campo	Detalle
Materia	40032 - Paradigmas de la Programacion
Practica	Programacion logica en Prolog
Lenguaje	Prolog
Documento PDF	index.pdf

1. Introduccion

En esta practica se trabajo con **Prolog**, un lenguaje basado en el paradigma de programacion logica. A diferencia de los lenguajes imperativos, en Prolog el programa se construye principalmente con hechos, reglas y consultas.

Los ejercicios desarrollados representan problemas pequenos donde se declara conocimiento y despues se consulta al interprete para obtener respuestas. La practica incluye ejemplos basicos, operaciones con numeros, circuitos resistivos, segmentos de recta, Torre de Hanoi, lista enlazada y el problema clasico del mono y el platano.

2. Objetivo

Aplicar los elementos principales de Prolog mediante bases de conocimiento y reglas que permitan resolver problemas usando hechos, predicados, variables, recursividad y busqueda logica.

Objetivos especificos:

- Declarar hechos y reglas simples.
- Realizar consultas sobre una base de conocimiento.
- Usar variables para obtener respuestas generales.
- Aplicar recursividad en problemas como Torre de Hanoi y listas enlazadas.
- Representar estados y acciones en el problema del mono y el platano.
- Modelar operaciones matematicas simples mediante reglas logicas.

3. Estructura de archivos

La practica esta formada por los siguientes programas en Prolog:

Archivo	Descripcion
<code>basicos.pl</code>	Define hechos sobre personas y reglas para identificar si una persona esta ocupada.
<code>max.pl</code>	Calcula el maximo y minimo entre dos numeros.
<code>circuitos_resistivos.pl</code>	Calcula resistencia equivalente en serie y paralelo.
<code>segmentos_recta.pl</code>	Representa puntos, segmentos, longitudes, pendientes y puntos medios.
<code>torre_hanoi.pl</code>	Resuelve movimientos de la Torre de Hanoi usando recursion.
<code>lista_enlazada.pl</code>	Modela una lista enlazada con nodos y operaciones basicas.
<code>mono_platano.pl</code>	Representa estados y acciones para resolver el problema del mono y el platano.

4. Desarrollo

4.1 Hechos y reglas basicas

El archivo `basicos.pl` muestra la estructura elemental de Prolog: primero se declaran hechos y despues reglas que dependen de esos hechos.

```
persona(juan).
persona(maria).

estudia(juan).
trabaja(maria).

ocupado(X) :- estudia(X).
ocupado(X) :- trabaja(X).
```

Con esta base de conocimiento se puede consultar si una persona esta ocupada:

```
?- ocupado(juan).
?- ocupado(maria).
?- ocupado(X).
```

4.2 Maximo y minimo

El archivo `max.pl` utiliza comparaciones aritmeticas para determinar el valor mayor o menor entre dos numeros.

```
maximo(X, Y, X) :- X >= Y.
maximo(X, Y, Y) :- X < Y.
```

```
minimo(X, Y, X) :- X =< Y.  
minimo(X, Y, Y) :- X > Y.
```

Ejemplos de consulta:

```
?- maximo(8, 3, X).  
?- minimo(8, 3, X).
```

4.3 Circuitos resistivos

En `circuitos_resistivos.pl` se declararon resistencias y reglas para calcular la resistencia equivalente en serie y en paralelo.

```
serie(R1, R2, Req) :-  
    Req is R1 + R2.  
  
paralelo(R1, R2, Req) :-  
    Req is (R1 * R2) / (R1 + R2).
```

Tambien se agregaron consultas por nombre de resistencia:

```
serie_nombre(A, B, Req) :-  
    resistencia(A, R1),  
    resistencia(B, R2),  
    serie(R1, R2, Req).
```

Ejemplos de consulta:

```
?- serie(10, 20, X).  
?- paralelo(10, 20, X).  
?- serie_nombre(r1, r2, X).
```

4.4 Segmentos de recta

El archivo `segmentos_recta.pl` representa puntos en el plano cartesiano y segmentos formados por esos puntos.

```
punto(a, 0, 0).  
punto(b, 4, 0).  
punto(c, 4, 3).  
punto(d, 0, 3).
```

```
segmento(ab, a, b).  
segmento(bc, b, c).
```

Una de las reglas calcula la longitud entre dos puntos:

```
longitud(P1, P2, L) :-  
    punto(P1, X1, Y1),  
    punto(P2, X2, Y2),  
    DX is X2 - X1,  
    DY is Y2 - Y1,  
    L is sqrt((DX * DX) + (DY * DY)).
```

Ejemplos de consulta:

```
?- longitud(a, c, L).  
?- punto_medio(a, c, X, Y).  
?- horizontal(a, b).  
?- vertical(b, c).
```

4.5 Torre de Hanoi

El archivo `torre_hanoi.pl` resuelve el problema de la Torre de Hanoi mediante recursion. El caso base mueve un solo disco y el caso recursivo divide el problema en tres pasos.

```
hanoi(N) :-  
    mover(N, izquierda, derecha, centro).  
  
mover(1, Origen, Destino, _) :-  
    write('Mover disco de '),  
    write(Origen),  
    write(' a '),  
    write(Destino),  
    nl.  
  
mover(N, Origen, Destino, Auxiliar) :-  
    N > 1,  
    M is N - 1,  
    mover(M, Origen, Auxiliar, Destino),  
    mover(1, Origen, Destino, Auxiliar),  
    mover(M, Auxiliar, Destino, Origen).
```

Ejemplo de consulta:

```
?- hanoi(3).
```

4.6 Lista enlazada

En `lista_enlazada.pl` se modelo una lista enlazada usando la estructura `nodo(Valor, Siguiete)`, donde `nil` representa el final de la lista.

```
lista_ejemplo(nodo(10, nodo(20, nodo(30, nil)))).

cabeza(nodo(X, _), X).

cola(nodo(_, Resto), Resto).
```

Tambien se agregaron reglas recursivas para buscar elementos, calcular longitud e insertar al inicio o al final.

```
pertenece(X, nodo(X, _)).
pertenece(X, nodo(_, Resto)) :-
    pertenece(X, Resto).

longitud(nil, 0).
longitud(nodo(_, Resto), N) :-
    longitud(Resto, N1),
    N is N1 + 1.
```

Ejemplos de consulta:

```
?- lista_ejemplo(L), cabeza(L, X).
?- lista_ejemplo(L), pertenece(20, L).
?- lista_ejemplo(L), longitud(L, N).
```

4.7 El mono y el platano

El archivo `mono_platano.pl` representa el problema clasico donde un mono debe moverse, empujar una caja, subirse a ella y tomar el platano.

El estado se representa con cuatro datos:

```
estado(PosMono, PosCaja, MonoEnCaja, TienePlatano)
```

La base incluye un estado inicial y un estado final:

```
estado_inicial(estado(puerta, ventana, no, no)).
estado_final(estado(_, _, si, si)).
```

Las acciones modifican el estado del problema:

```
accion(  
    estado(centro, centro, si, no),  
    tomar_platano,  
    estado(centro, centro, si, si)  
).
```

La solución se obtiene buscando una lista de acciones válidas:

```
solucion(Pasos) :-  
    estado_inicial(Inicio),  
    resolver(Inicio, Pasos, _, [Inicio]).
```

Ejemplo de consulta:

```
?- solucion(Pasos).
```

5. Evidencia

La evidencia de esta práctica corresponde a los archivos fuente de Prolog y a las consultas documentadas para probar cada programa.

Consultas principales documentadas:

```
?- ocupado(X).  
?- maximo(8, 3, X).  
?- serie_nombre(r1, r2, X).  
?- longitud(a, c, L).  
?- hanoi(3).  
?- lista_ejemplo(L), longitud(L, N).  
?- solucion(Pasos).
```

6. Resultados

Como resultado de la práctica se cuenta con una colección de programas Prolog que cubren distintos usos del paradigma lógico:

- Hechos y reglas simples para consultar información.
- Predicados aritméticos para máximo, mínimo y resistencias.
- Representación de puntos y segmentos mediante hechos.
- Recursión aplicada a Torre de Hanoi y listas enlazadas.

- Búsqueda de acciones validas a partir de estados en el problema del mono y el platano.

Los archivos quedan preparados para cargarse en un interprete Prolog y ejecutarse mediante las consultas indicadas en cada seccion.

7. Conclusion

La practica permitio reforzar la forma de pensar de la programacion logica: en lugar de describir paso a paso todas las instrucciones, se declaran relaciones, condiciones y reglas para que Prolog encuentre respuestas. Los ejercicios muestran como este enfoque puede aplicarse tanto a problemas matematicos sencillos como a problemas recursivos y de busqueda por estados.